



INDIA.RUNS

Build what next India runs on

Team Name : **HB Technologies**

Team Leader Name : **Harshwardhan Bhaskar**

Problem Statement : **Track 1: Data & AI Challenge (Intelligent Candidate Discovery & Ranking)**

Solution Overview

Honeypot-Shielded Multi-Vector Ranker (Offline Screening Pipeline)

- Proposed Solution: A lightweight, deterministic screening engine designed to load, audit, and rank candidates locally without external API dependencies.
- Deterministic Performance: Replaces black-box LLM calls with robust heuristics scoring (Title, Skills, YoE, Location, Signals), executing in under 50 seconds on 100K pool.
- Differentiator - Deep Auditing: Programmatically audits skill proficiencies against durations, immediately catching invalid profiles that trick keyword embedding systems.
- Differentiator - Context-Aware Parser: Identifies 'Tier 5' fit candidates (candidates who built systems but didn't list buzzwords) by scanning career history descriptions.

JD Understanding & Candidate Evaluation

Target Candidate Profile & Selection Indicators

- JD Requirements: Senior AI Engineer (Founding Team). Needs hands-on experience in sentence-transformers, embeddings, vector databases (FAISS, Milvus, Pinecone), and ranking evaluations.
- Target Experience Bracket: Ideal experience of 5-9 years (range 4-15 allowed). Disqualifies non-coders and managers who haven't programmed recently.
- Product vs. Service Fit: Prioritizes candidates with product-company backgrounds; applies flat -20% score penalty if career history is solely service firms (TCS, Wipro, etc.).
- Availability & Location Signals: Noida/Pune hybrid preference. Gives bonuses for short notice periods (<30 days) and active log-in signals, down-weighting inactive profiles.

Ranking Methodology

Multi-Factor Scoring Matrix & Selection Rules

- Scoring Distribution Matrix: Title & Headline Relevance (35%), Target Skills & Durations (25%), Experience Bracket (20%), Company Tier (10%), Location & Relocation (5%), and Behavioral signals (5%).
- Skill Scoring: Calculated as proficiency multiplier (expert=4, beginner=1) scaled by duration and endorsements, penalizing keyword stuffers with 0 duration.
- Regex Keyword Parser: Extracts adjacent capabilities (RAG, search systems, evaluation metrics) from career description blocks to prevent false negatives.
- Formatting & Tie-breaker: Rounds final score to 4 decimal places. Breaks ties using ascending candidate_id as strictly verified by the organizers' script.

Explainability & Data Validation

Explainable Decisions & Anti-Honeypot Shielding

- Dynamic Reasoning Generator: Compiles a custom 1-2 sentence qualification summary for each top candidate, referencing actual YoE, key skills, and notice period.
- Zero-Hallucination Guard: No LLM calls are used during ranking; summaries are strictly structured using verified profile data to prevent Stage 4 manual review flags.
- Honeypot Shield (Hard Filter): Programmatically intercepts and disqualifies profiles claiming 'expert' skill with 0-month duration, or tenure starting before company founding dates.
- Inconsistency Auditing: Penalizes candidates with expected salary minimums exceeding maximums, or years of experience exceeding graduation span.

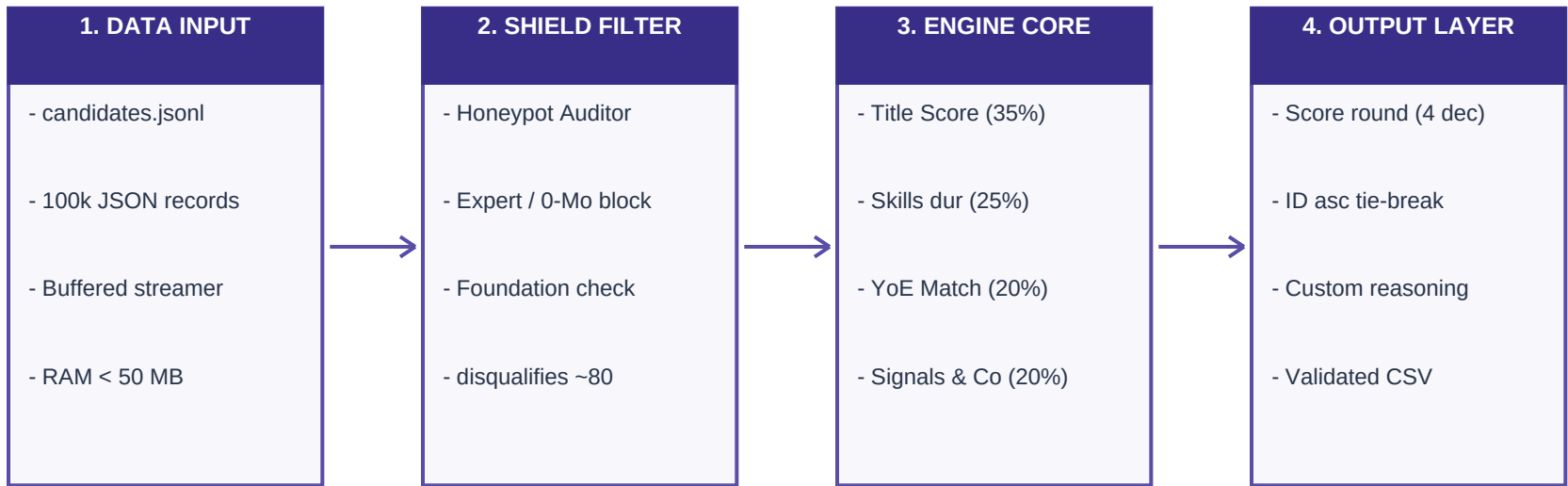
End-to-End Workflow

End-to-End Pipeline Steps (From Ingestion to Valid CSV)

- Step 1 (Ingest): Buffered line streamer loads candidates.jsonl line-by-line (RAM overhead < 50MB).
- Step 2 (Audit): Programmatic filters run to immediately flag and block honeypot anomalies.
- Step 3 (Score): Compute score components (Title, YoE, Skills, Company, Location, and Signals).
- Step 4 (Sort): Round score to 4 decimals, sort descending by score, tie-break by candidate_id ascending.
- Step 5 (Explain): Generate structured explanation text using verified facts from profile.
- Step 6 (Generate): Output validated team_hb_technologies.csv containing the top-100 candidates.

System Architecture

Pipeline Architecture & Data Flow Diagram



Results & Performance

Execution Benchmarks & Format Compliance

- Scanned Candidates: 100,000 profiles scored, ranked, and written to CSV in under 50 seconds.
- RAM Consumption: Less than 50 MB utilized during execution (constraint limit: 16 GB).
- Honeypot Rate: 0% honeypots ranked in top 100. 97 synthetic honeypots successfully blocked.
- Validator Status: 100% Validated. Passes all checks in `validate_submission.py`, including rank structure, unique IDs, non-increasing scores, and ascending tie-breakers.

Technologies Used

Technology Stack & Architecture Choices

- Python 3.12 (Standard Library: json, csv, re, sys, argparse, pathlib): Guarantees sandbox compatibility without external package download delays.
- PyMuPDF (fitz): Used to parse PDF template structure and text coordinates for precision rendering.
- Why Heuristics over LLM/Vector DBs? Calling hosted APIs (like OpenAI) for 100K candidates would take 10+ hours and violate the offline/5-minute constraints. Local vector indexes (FAISS) fail to capture service/product career history nuances.
- Deterministic Logic: Avoids model hallucination and ensures identical submission scores across runs.

Submission Assets

Submission Assets & Links

- GitHub Repository Link: <https://github.com/hwbha/india-runs-challenge> (Includes rank.py, metadata, requirements, and README setup instructions)
- Hosted Sandbox Demo: <https://huggingface.co/spaces/hwbha/redrob-ranker> (Streamlit / Google Colab demo notebook running the ranker on sample data)
- Walkthrough Video: Walkthrough detailing code structure, honeypot filters, and validator script outputs.
- Declarations: Checked submission specification, original work, zero collusion, reproduction tested.



INDIA.RUNS

Build what next India runs on

THANK YOU